6263-ITAI-1371-Intro to Machine Learning

Midterm Exam: Reflection & Contribution Journal

Student Name: Cuong Dang (Student ID: W217887095)

Group Members: Astríd Lorenzana, Jonathan Aldana, Dana Bolden, Ruben Nuno

Group Designation: Group 1 (ML_1371_12321 1)

Submission Date: July 11, 2026

I. Project Objective & Core Reflection

The operational objective of this midterm project is to establish a rigorous, leak-free exploratory data analysis (EDA) and preprocessing pipeline using the 7,043-row Kaggle Telco Customer Churn dataset. This clean data environment directly serves as the mathematical foundation for our upcoming Final Exam, where we will train machine learning classification frameworks (such as Logistic Regression and Random Forests).

From a predictive engineering standpoint, our goal is optimizing the model's Recall score. Maximizing Recall ensures that high-risk churning subscribers are flagged accurately, empowering corporate retention strategies to systematically minimize business revenue deficits.

II. Assignment Execution Status

To ensure the team has a stable and approved framework well ahead of the deadline, I have taken the initiative to develop the initial baseline data pipeline, programmatic architecture, and visualization layouts based on my dataset selection. To maintain total professional transparency and peer collaboration, I have shared these completed code blocks and documentation files directly with all group members inside our communication channel as a foundational starting point. This leaves our repository fully open for all team members to review, adapt, modify, or append their own unique project features, code modifications, or analytical reflections before the final July 11 deadline.

III. Detailed Individual Contributions (Cuong Dang)

1. Dataset Selection & Project Architecture

* Identified, downloaded, and validated the 7,043-row "Telco Customer Churn" dataset from Kaggle, satisfying the professor's strict constraint of exceeding 2,000 rows.

* Written the comprehensive 1-page technical project proposal detailing the classification objectives, feature layouts, and target business optimization targets.

2. Programmatic Preprocessing Pipeline

* Structured the baseline Jupyter Notebook (‘Group01_ITAI1371_CD_Midterm_eda.ipynb’) and integrated the ‘kagglehub’ cloud API to automate remote dataset fetching to prevent local directory mapping errors

* Implemented a strict, programmatic 70% Training / 30% Testing data matrix partition using Python to guarantee zero downstream data leakage.

* Isolated the 30% testing array directly to an untouched standalone CSV file (‘untouched_test_data.csv’).

* Coded automated data cleaning blocks, replacing hidden missing structural spaces inside the continuous variable "TotalCharges" using the training matrix median value.

* Engineered One-Hot Encoded sparse matrices for categorical text feature rows (‘gender’,‘Contract’,‘PaymentMethod’) to prepare them for mathematical model training.

* Mitigated severe class imbalance within the target label by coding a minority class upsampling framework to systematically balance the dataset uniform distribution.

* Standardized features via ‘StandardScaler’ to bring disparate continuous dimensions into a uniform scaled space.

3. Visual Exploratory Data Analysis & Chart Isolation

* Built Matplotlib and Seaborn visualization subplots executed exclusively on the 70% training block to profile data shapes without compromising test validity.

* Generated and exported visual validation files ('before_preprocessing_plots.png' and 'after_preprocessing_plots.png') proving successful class balancing and scaling transformations.

Astríd Lorenzana's contribution:

With the help of Cuong, I was guided into how the analysis should look like for a seaborn heatmap. I had trouble with how to facilitate even more the code considering it looked really clean already with Cuong's example. Coding is pretty difficult to me considering it is a lot of information so I had to look through other examples, access resources to understand how the code can be simplified even further and understanding it step by step, I also looked into my other course where I am beginning to learn about coding topics.

Lots of tries and failures along the way. I'm a bit clumsy, so I would accidentally delete the full code, or pieces of the code that were important that made the code flunk when missing. I would add a statement in hopes of improving the code but once again failed. Eventually, I found out that a way to make the code even cleaner was by defining a function, looking at it step by step and seeing how it would work in my situation with the data our group has and got it to work. I implemented the example given by our teammate Cuong, examples from other resources, and what I learned from my other course. (astrid_correlation_heatmap.png)

Jonathan Aldana: Individual Reflections

Replicating the pipeline on my end demonstrated how critical script-based data ingestion is for collaborative data science, proving that the group's code is robust and yields identical results across different development setups. Verifying the train/test split boundary taught me how easily

data leakage can corrupt validation metrics, reinforcing why isolating the 2,113 test rows as an untouched standalone CSV is vital for real-world model tracking. Furthermore, working through the preprocessing blocks firsthand showed me how raw distribution anomalies directly bias machine learning models, and how statistical tools like median imputation, scaling, and over-sampling create a balanced mathematical landscape for classification weights

Contributions

*Pipeline Replication & Verification: Replicated the entire Python data pipeline locally to test for reproducibility, running every code cell sequentially to verify that my environment generated the exact same baseline results, data dimensions, and class balancing outputs as Cuong.

*Pipeline Validation & Audit: Followed through the data preprocessing and splitting logic, checking that the 70/30 train/test split, missing value median imputation, and feature scaling steps made mathematical sense according to the assignment guidelines.

*Categorical Correlation Code Addition: Authored and appended a unique exploratory correlation script at the bottom of the notebook to track the engineered dummy columns against the target matrix, plotting and exporting the final bar chart as jon_categorical_correlation.png.